LAPORAN UJIAN TENGAH SEMESTER

SOAL TIPE C (TI – C3)

MATA KULIAH PEMBELAJARAN MESIN PRAKTIKUM

Oleh:

Alisya Aninda Safwa	434241032
Nadine Nazwa Andina	434241035
Cindy Tiara Anastasya	434241037

D-IV TEKNIK INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS AIRLANGGA

2026/2027

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR GAMBAR **4**
DAFTAR TABEL **5**
BAB I **6**
PENDAHULUAN **6**
1.1 Tujuan 6
1.2 Dataset 6
BAB II **8**
PROSES PENGERJAAN **8**
2.1 Flowchart Proses 8
2.2 Penjelasan Alur Proses 8
BAB III **10**
HASIL DAN PEMBAHASAN **10**
3.1 Input Data dan Data Understanding 10
3.1.1 Input Dataset 10
3.1.3 Statistik Deskriptif 12
3.1.4 Distribusi Target 12
3.2 Preprocessing 14
3.2.1 Pemeriksaan Missing Value 14
3.2.2 Pemeriksaan Data Duplikat 15
3.2.3 Deteksi Outlier 15
3.2.4 Handling Outlier 17
3.3 Transformasi 18
3.3.1 Penghapusan Kolom 18
3.3.2 Encoding Data Kategorikal 18
3.3.3 Pemisahan Fitur dan Target 19
3.3.4 Normalisasi MinMaxScaler 20
3.4 Split Data 20

3.4.1 Pembagian Data Training dan Testing 20
3.4.2 Distribusi Data Training dan Testing 21
3.4.3 Baseline Data 22
3.5 Oversampling 23
3.5.1 Metode Oversampling 23
3.5.2 Distribusi Kelas Sebelum Oversampling 24
3.5.3 Implementasi Oversampling 25
3.5.4 Distribusi Kelas Setelah Oversampling 26
3.5.5 Training Decision Tree 27
3.5.6 Testing dan Evaluasi 27
3.6 Undersampling 29
3.6.1 Metode Undersampling 29
3.6.2 Distribusi Kelas Sebelum Undersampling 30
3.6.3 Implementasi Undersampling 30
3.6.4 Distribusi Kelas Setelah Undersampling 32
3.6.5 Training Decision Tree 32
3.6.6 Testing dan Evaluasi 33
3.7 Perbandingan Hasil 36
3.7.1 Perbandingan Confusion Matrix 36
3.7.2 Perbandingan Accuracy, Precision, Recall, dan F1-Score 37
3.7.3 Pembahasan Hasil 38
BAB IV 40
KESIMPULAN 40

DAFTAR GAMBAR

Gambar 2. 1 Flowchart Proses 8
Gambar 3. 1 Distribusi Kelas Awal 13
Gambar 3. 2 Boxplot Sebelum Handling Outlier 16
Gambar 3. 3 Boxplot Setelah Handling Outlier 18
Gambar 3. 4 Grafik Distribusi 22
Gambar 3. 5 Diagram Perubahan Distribusi 26
Gambar 3. 6 Confusion Matrix - Oversampling 28
Gambar 3. 7 Grafik Perbandingan 32
Gambar 3. 8 Confusion Matrix Undersampling 34
Gambar 3. 9 Confusion Matrix Oversampling 36
Gambar 3. 10 Confusion Matrix Undersampling 37

DAFTAR TABEL

Tabel 2. 1 Penjelasan Alur Proses 9
Tabel 3. 1 Kolom dan Jenis Data pada Dataset 11
Tabel 3. 2 Distribusi Kelas Sebelum Oversampling 25
Tabel 3. 3 Distribusi Kelas Setelah Oversampling 26
Tabel 3. 4 Metrik Evaluasi 28
Tabel 3. 5 Distribusi kelas sebelum Undersampling 30
Tabel 3. 6 Perbandingan hasil perubahan distribusi 32
Tabel 3. 7 Metrik Evaluasi Secara Keseluruhan 35
Tabel 3. 8 Ringkasan Komparasi Metrik Evaluasi 37

BAB I

PENDAHULUAN

1.1 Tujuan

Laporan ini disusun sebagai salah satu bentuk pemenuhan dan penilaian Ujian Tengah Semester (UTS) pada mata kuliah Pembelajaran Mesin. Adapun tujuan dari proses pengolahan dan klasifikasi data ini adalah sebagai berikut:

1. Melakukan *data understanding* terhadap dataset yang digunakan.
2. Melakukan *preprocessing* untuk mempersiapkan data sebelum proses klasifikasi.
3. Melakukan transformasi data agar dapat digunakan dalam proses pembelajaran mesin.
4. Membagi data menjadi data *training* dan *testing* dengan proporsi 80:20.
5. Menerapkan metode oversampling dan undersampling untuk menangani *imbalanced data*.
6. Menerapkan algoritma Decision Tree untuk melakukan klasifikasi.
7. Mengevaluasi hasil klasifikasi menggunakan Confusion Matrix, accuracy, precision, recall, dan F1-score.
8. Membandingkan hasil klasifikasi dari pendekatan oversampling dan undersampling.

1.2 Dataset

Dataset yang digunakan dalam pengerjaan ini adalah Machine Predictive Maintenance Classification yang diperoleh dari Kaggle (https://www.kaggle.com/datasets/shivamb/machine-predictive-maintenance-classification). Dataset tersebut berisi 10.000 data yang menggambarkan kondisi operasional mesin dan digunakan untuk melakukan klasifikasi terhadap kondisi kegagalan mesin.

Dataset memiliki beberapa atribut yang merepresentasikan karakteristik mesin, antara lain Air Temperature [K], Process Temperature [K], Rotational Speed [rpm], Torque

[Nm], dan Tool Wear [min]. Selain fitur numerik tersebut, terdapat atribut kategorikal Type yang memiliki kategori L, M, dan H.

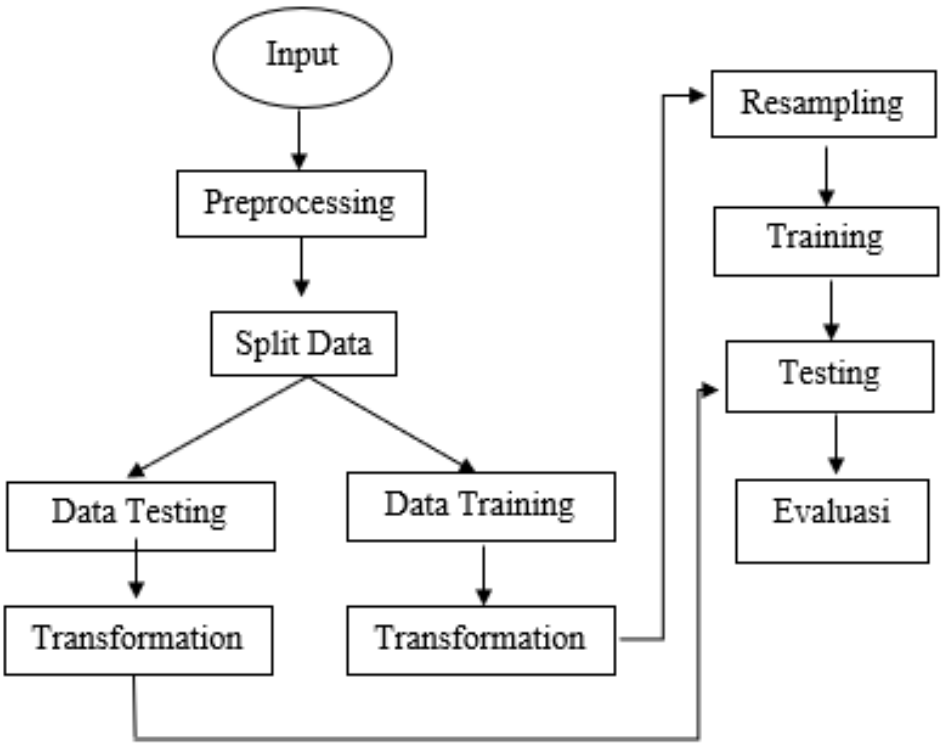
Target klasifikasi terdapat pada kolom Target, dengan nilai 0 menunjukkan kondisi mesin normal dan nilai 1 menunjukkan terjadinya kegagalan mesin. Dataset memiliki distribusi kelas yang tidak seimbang (*imbalanced data*), sehingga diperlukan penanganan lebih lanjut pada data training menggunakan metode oversampling dan undersampling.

BAB II

PROSES PENGERJAAN

2.1 Flowchart Proses

Flowchart yang digunakan dalam pengerjaan ini mengacu pada flowchart yang tercantum pada soal UTS. Flowchart tersebut menggambarkan tahapan proses mulai dari input data, preprocessing, transformasi, pembagian data, resampling, klasifikasi, hingga evaluasi.



Gambar 2. 1 Flowchart Proses

2.2 Penjelasan Alur Proses

Alur pengerjaan mengikuti tahapan yang terdapat pada flowchart soal UTS, mulai dari input data hingga evaluasi hasil klasifikasi. Penjelasan setiap tahapan adalah sebagai berikut:

No.	Tahapan	Penjelasan
1	Input	Dataset dimasukkan dan digunakan sebagai data awal untuk proses pengolahan.
2	Preprocessing	Melakukan pemeriksaan terhadap missing value, data duplikat, dan outlier, kemudian melakukan handling apabila ditemukan.
3	Transformation	Melakukan transformasi data agar sesuai untuk digunakan dalam proses pembelajaran mesin.
4	Split Data	Membagi data menjadi 80% data training dan 20% data testing.
5	Resampling	Melakukan oversampling dan undersampling untuk menangani ketidakseimbangan kelas. Resampling hanya dilakukan pada data training.
6	Training	Melatih model klasifikasi menggunakan algoritma Decision Tree berdasarkan data training hasil resampling.
7	Testing	Menguji model menggunakan data testing yang tidak melalui proses resampling.
8	Evaluasi	Mengevaluasi hasil klasifikasi menggunakan Confusion Matrix, accuracy, precision, recall, dan F1-score.

Tabel 2. 1 Penjelasan Alur Proses

BAB III

HASIL DAN PEMBAHASAN

3.1 Input Data dan Data Understanding

3.1.1 Input Dataset

Tahap pengerjaan diawali dengan memasukkan dataset ke dalam program. Dataset yang digunakan adalah Machine Predictive Maintenance Classification yang diperoleh dari Kaggle dan disimpan dalam file predictive_maintenance.csv. Dataset ini digunakan untuk melakukan klasifikasi kondisi mesin berdasarkan data operasional yang tersedia.

Dataset dimuat menggunakan fungsi pd.read_csv(), dengan lokasi dataset yang telah ditentukan pada file konfigurasi config.py. File konfigurasi tersebut digunakan untuk menyimpan berbagai pengaturan yang dibutuhkan selama proses pengolahan data. Proses pemuatan dataset dilakukan melalui fungsi load_dataset() pada init_dataset.py.

```
 9   def load_dataset(path=config.DATA_PATH) -> pd.DataFrame:
10       return pd.read_csv(path)
```

```
============================================================
1. INIT DATASET & DATA UNDERSTANDING
============================================================

--- Ukuran dan 5 baris pertama ---
Ukuran data (baris, kolom): (10000, 10)
   UDI Product ID Type  Air temperature [K]  Process temperature [K]  Rotational speed [rpm]  Torque [Nm]  Tool wear [min]  Target Failure Type
0    1     M14860    M                298.1                    308.6                    1551         42.8                0       0   No Failure
1    2     L47181    L                298.2                    308.7                    1408         46.3                3       0   No Failure
2    3     L47182    L                298.1                    308.5                    1498         49.4                5       0   No Failure
3    4     L47183    L                298.2                    308.6                    1433         39.5                7       0   No Failure
4    5     L47184    L                298.2                    308.7                    1408         40.0                9       0   No Failure
```

3.1.2 Informasi Dataset

Tahap *data understanding* dilakukan untuk memperoleh gambaran awal mengenai struktur dataset sebelum data diproses lebih lanjut. Pemeriksaan dilakukan menggunakan info() untuk melihat nama kolom, jumlah data, dan tipe data yang terdapat pada dataset.

```python
def understand(df: pd.DataFrame) -> None:
    """Data understanding: ukuran, tipe, statistik, distribusi target & Type."""
    subbanner("Ukuran dan 5 baris pertama")
    print("Ukuran data (baris, kolom):", df.shape)
    print(df.head().to_string())

    subbanner("Info kolom dan tipe data")
    df.info()       You, 2 days ago • init dataset, preprocessing, transformation
```

```
--- Info kolom dan tipe data ---
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   UDI                     10000 non-null  int64
 1   Product ID              10000 non-null  str
 2   Type                    10000 non-null  str
 3   Air temperature [K]     10000 non-null  float64
 4   Process temperature [K] 10000 non-null  float64
 5   Rotational speed [rpm]  10000 non-null  int64
 6   Torque [Nm]             10000 non-null  float64
 7   Tool wear [min]         10000 non-null  int64
 8   Target                  10000 non-null  int64
 9   Failure Type            10000 non-null  str
dtypes: float64(3), int64(4), str(3)
memory usage: 781.4 KB
```

Dataset memiliki beberapa jenis atribut yang terdiri atas kolom identitas, kategorikal, numerik, dan target klasifikasi. Rincian atribut yang terdapat dalam dataset ditunjukkan pada Tabel 3.1.

Kolom	Jenis Data	Keterangan
UDI	Identitas	Nomor identitas data
Product ID	Identitas	Identitas produk
Type	Kategorikal	Kategori produk: L, M, dan H
Air temperature [K]	Numerik	Suhu udara
Process temperature [K]	Numerik	Suhu proses
Rotational speed [rpm]	Numerik	Kecepatan rotasi mesin
Torque [Nm]	Numerik	Torsi mesin
Tool wear [min]	Numerik	Waktu penggunaan alat
Target	Target	Label klasifikasi: 0 = Normal, 1 = Failure
Failure Type	Kategorikal	Jenis kegagalan mesin

Tabel 3. 1 Kolom dan Jenis Data pada Dataset

Selain struktur data, distribusi atribut Type juga diperiksa untuk mengetahui jumlah data pada masing-masing kategori. Informasi mengenai struktur dataset tersebut menjadi

dasar untuk menentukan fitur yang akan dipertahankan, ditransformasikan, maupun dikeluarkan pada tahap berikutnya.

3.1.3 Statistik Deskriptif

Setelah struktur dataset diketahui, dilakukan pemeriksaan statistik deskriptif untuk memperoleh gambaran mengenai karakteristik fitur numerik. Pemeriksaan dilakukan menggunakan fungsi describe() yang menghasilkan informasi berupa jumlah data, nilai rata-rata, standar deviasi, nilai minimum, kuartil, dan nilai maksimum.

```
subbanner("Statistik deskriptif (kolom numerik)")
print(df.describe().round(2).T.to_string())
```

```
--- Statistik deskriptif (kolom numerik) ---
                         count     mean      std     min      25%     50%      75%      max
UDI                    10000.0  5000.50  2886.90     1.0  2500.75  5000.5  7500.25  10000.0
Air temperature [K]    10000.0   300.00     2.00   295.3   298.30   300.1   301.50    304.5
Process temperature [K]10000.0   310.01     1.48   305.7   308.80   310.1   311.10    313.8
Rotational speed [rpm] 10000.0  1538.78   179.28  1168.0  1423.00  1503.0  1612.00   2886.0
Torque [Nm]            10000.0    39.99     9.97     3.8    33.20    40.1    46.80     76.6
Tool wear [min]        10000.0   107.95    63.65     0.0    53.00   108.0   162.00    253.0
Target                 10000.0     0.03     0.18     0.0     0.00     0.0     0.00      1.0
```

3.1.4 Distribusi Target

Selain karakteristik fitur, distribusi variabel target juga diperiksa karena Target merupakan label yang akan digunakan dalam proses klasifikasi. Nilai 0 menunjukkan kondisi mesin normal, sedangkan nilai 1 menunjukkan kondisi mesin mengalami kegagalan.

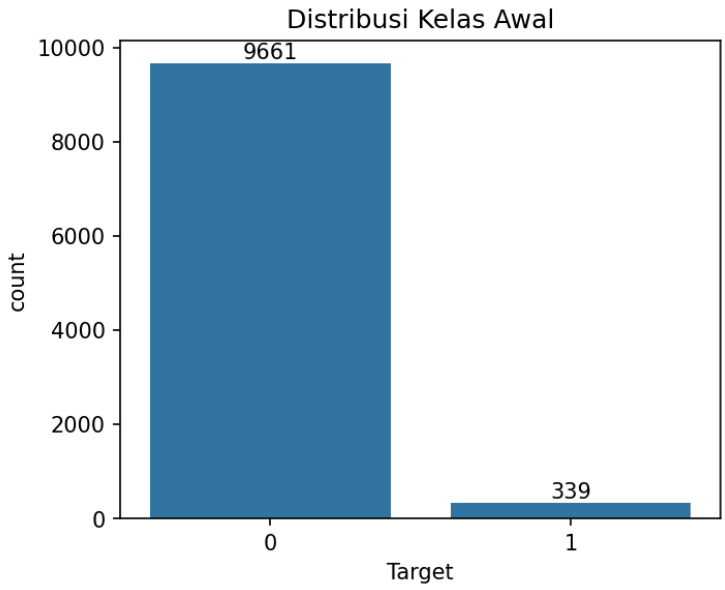
```
subbanner(f"Distribusi kelas {config.TARGET} (0 = normal, 1 = gagal)")
counts = df[config.TARGET].value_counts()
persen = (df[config.TARGET].value_counts(normalize=True) * 100).round(2)
print(pd.DataFrame({"jumlah": counts, "persen": persen}).to_string())

subbanner(f"Distribusi kolom {config.TYPE_COLUMN}")
print(df[config.TYPE_COLUMN].value_counts().to_string())

for col in config.LEAKAGE_COLUMNS:
    if col in df.columns:
        subbanner(f"Pemeriksaan kebocoran label: {col} vs {config.TARGET}")
        print(pd.crosstab(df[col], df[config.TARGET]).to_string())
```

Hasil pemeriksaan distribusi target digunakan untuk mengetahui keseimbangan kelas pada dataset. Perbedaan jumlah antara kelas normal dan failure selanjutnya menjadi pertimbangan dalam penanganan *imbalanced data* pada data training.

Untuk memberikan gambaran secara visual, distribusi kelas target juga divisualisasikan menggunakan *count plot*.



Gambar 3. 1 Distribusi Kelas Awal

Selain distribusi target, dilakukan pemeriksaan terhadap Failure Type melalui *cross-tabulation* dengan Target. Pemeriksaan ini dilakukan karena Failure Type berkaitan langsung dengan informasi kegagalan mesin dan berpotensi memberikan informasi label kepada model.

```
--- Pemeriksaan kebocoran label: Failure Type vs Target ---
Target                      0    1
Failure Type
Heat Dissipation Failure    0  112
No Failure               9643    9
Overstrain Failure          0   78
Power Failure               0   95
Random Failures            18    0
Tool Wear Failure           0   45
```

Berdasarkan pemeriksaan tersebut, Failure Type tidak digunakan sebagai fitur dalam proses klasifikasi. Bersama dengan UDI dan Product ID, kolom tersebut akan dikeluarkan pada tahap transformasi. Setelah karakteristik awal data diketahui, dataset

selanjutnya masuk ke tahap preprocessing untuk memastikan kondisi data siap digunakan pada proses berikutnya.

3.2 Preprocessing

3.2.1 Pemeriksaan Missing Value

Pemeriksaan pertama dilakukan terhadap *missing value* untuk mengetahui apakah terdapat nilai kosong pada setiap kolom dataset. Pemeriksaan dilakukan menggunakan fungsi check_missing(), sedangkan fungsi handle_missing() disediakan sebagai mekanisme penanganan apabila ditemukan nilai kosong.

```python
# Missing value
def check_missing(df: pd.DataFrame) -> int:
    subbanner("Cek missing value")
    missing = df.isnull().sum()
    print(missing.to_string())
    total = int(missing.sum())
    print("Total missing value:", total)
    return total


def handle_missing(df: pd.DataFrame) -> pd.DataFrame:
    """Numerik -> median, kategorikal -> modus (hanya berjalan bila ada missing)."""
    df = df.copy()
    for col in df.columns[df.isnull().any()]:
        if pd.api.types.is_numeric_dtype(df[col]):
            df[col] = df[col].fillna(df[col].median())
        else:
            df[col] = df[col].fillna(df[col].mode()[0])
    return df      You, 2 days ago • init dataset, preprocessing, transformation and
```

```
======================================================================
2. PREPROCESSING
======================================================================

--- Cek missing value ---
UDI                        0
Product ID                 0
Type                       0
Air temperature [K]        0
Process temperature [K]    0
Rotational speed [rpm]     0
Torque [Nm]                0
Tool wear [min]            0
Target                     0
Failure Type               0
Total missing value: 0
Tidak ada missing value, tidak ada tindakan.
```

Berdasarkan hasil pemeriksaan, tidak ditemukan *missing value* pada dataset. Oleh karena itu, fungsi penanganan *missing value* tidak perlu dijalankan dan data dapat dilanjutkan ke pemeriksaan berikutnya tanpa perubahan pada bagian ini.

3.2.2 Pemeriksaan Data Duplikat

Pemeriksaan selanjutnya dilakukan terhadap data duplikat menggunakan fungsi df.duplicated(). Pemeriksaan ini bertujuan untuk mengetahui apakah terdapat baris data yang sama secara keseluruhan.

```python
# Duplikat
def check_duplicates(df: pd.DataFrame) -> pd.Series:
    """Mengecek baris data yang memiliki nilai duplikat."""
    subbanner("Cek data duplikat")
    mask = df.duplicated()
    print("Jumlah data duplikat:", int(mask.sum()))
    return mask

def handle_duplicates(df: pd.DataFrame, dup_mask: pd.Series) -> pd.DataFrame:
    if not dup_mask.any():
        print("Tidak ada duplikat, tidak ada tindakan.")
        return df
    df = df[~dup_mask].reset_index(drop=True)
    print("Duplikat dihapus. Ukuran data sekarang:", df.shape)
    return df
```

```
--- Cek data duplikat ---
Jumlah data duplikat: 0
Tidak ada duplikat, tidak ada tindakan.
```

Hasil pemeriksaan menunjukkan bahwa tidak terdapat data duplikat pada dataset. Dengan demikian, tidak dilakukan penghapusan baris dan jumlah data tetap dipertahankan untuk tahap berikutnya.

3.2.3 Deteksi Outlier

Setelah pemeriksaan duplikat selesai, tahap preprocessing dilanjutkan dengan mendeteksi *outlier* pada fitur numerik. Metode yang digunakan adalah Interquartile Range (IQR) dengan faktor 1.5. Metode ini digunakan untuk menentukan batas bawah dan batas atas pada masing-masing fitur numerik.

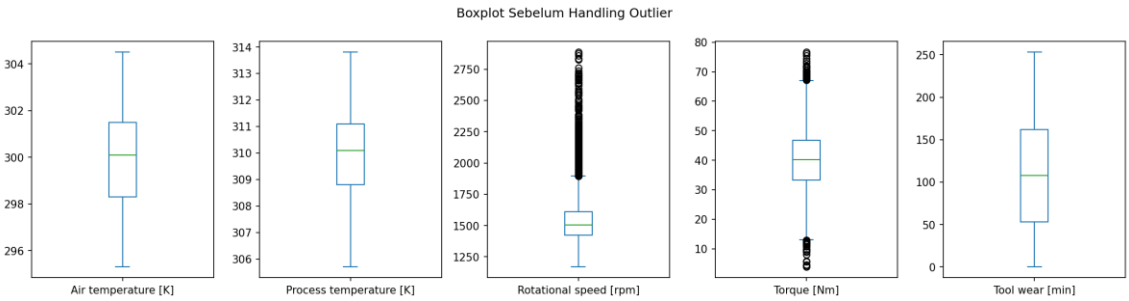
```python
# Outlier (IQR)
def compute_bounds(df: pd.DataFrame) -> dict:
    bounds = {}
    for col in config.NUMERIC_COLUMNS:
        q1, q3 = df[col].quantile(0.25), df[col].quantile(0.75)
        iqr = q3 - q1
        bounds[col] = (q1 - config.IQR_FACTOR * iqr, q3 + config.IQR_FACTOR * iqr)
    return bounds

def outlier_mask(df: pd.DataFrame, col: str, bounds: dict) -> pd.Series:
    low, high = bounds[col]
    return (df[col] < low) | (df[col] > high)


def detect_outliers(df: pd.DataFrame, bounds: dict, judul: str) -> pd.DataFrame:
    subbanner(judul)
    rows = []
    for col in config.NUMERIC_COLUMNS:
        mask = outlier_mask(df, col, bounds)
        rows.append({
            "Kolom": col,
            "Batas bawah": round(bounds[col][0], 2),
            "Batas atas": round(bounds[col][1], 2),
            "Jumlah outlier": int(mask.sum()),
            "Di antaranya gagal": int(df.loc[mask, config.TARGET].sum()),
        })
    summary = pd.DataFrame(rows)
    print(summary.to_string(index=False))
    return summary
```

```
--- Deteksi outlier (IQR, faktor 1.5) ---
                Kolom  Batas bawah  Batas atas  Jumlah outlier  Di antaranya gagal
    Air temperature [K]       293.50      306.30               0                   0
Process temperature [K]       305.35      314.55               0                   0
 Rotational speed [rpm]      1139.50     1895.50             418                  35
           Torque [Nm]        12.80       67.20              69                  62
       Tool wear [min]      -110.50      325.50               0                   0
[grafik tersimpan] 02_boxplot_sebelum.png
```

Hasil deteksi tersebut kemudian diperjelas menggunakan visualisasi boxplot sehingga kondisi distribusi fitur numerik sebelum dilakukan penanganan dapat diamati secara visual.



Gambar 3. 2 Boxplot Sebelum Handling Outlier

Berdasarkan hasil pemeriksaan tersebut, *outlier* yang ditemukan tidak dihapus dari dataset. Penanganan dilakukan menggunakan metode clipping agar jumlah observasi tetap dipertahankan

3.2.4 Handling Outlier

Berdasarkan hasil deteksi, *outlier* tidak dihapus dari dataset. Penanganan dilakukan menggunakan metode clipping, yaitu membatasi nilai yang berada di bawah batas minimum menjadi nilai batas bawah dan nilai yang berada di atas batas maksimum menjadi nilai batas atas. Dengan metode ini, baris data tetap dipertahankan sehingga tidak terjadi pengurangan jumlah observasi akibat penghapusan *outlier*.

Proses *clipping* dilakukan menggunakan fungsi clip_outliers() berdasarkan batas IQR yang telah dihitung sebelumnya.

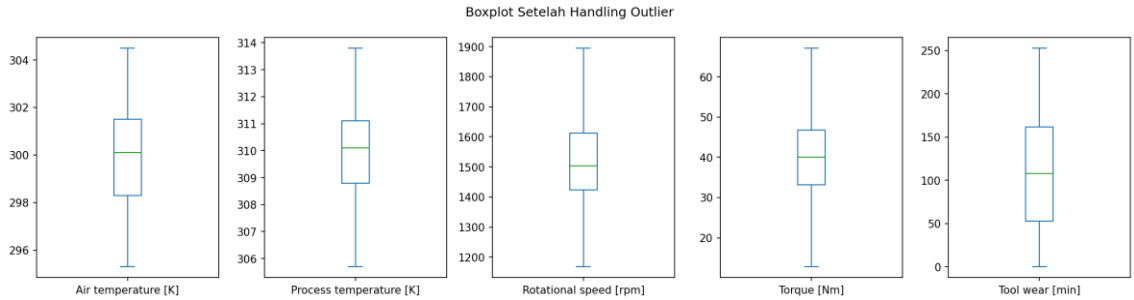
```
77  def clip_outliers(df: pd.DataFrame, bounds: dict) -> pd.DataFrame:
78      """Handling sesuai MD: clipping ke batas IQR (baris tidak dihapus)."""
79      df = df.copy()
80      for col, (low, high) in bounds.items():
81          df[col] = df[col].clip(lower=low, upper=high)
82      return df
```

Setelah proses *clipping* dilakukan, deteksi *outlier* dijalankan kembali untuk melihat kondisi data setelah penanganan. Pemeriksaan tersebut menggunakan batas IQR yang sama dengan pemeriksaan sebelumnya sehingga perubahan kondisi *outlier* dapat dibandingkan secara konsisten.

```
--- Outlier setelah clipping (batas IQR yang sama) ---
                Kolom  Batas bawah  Batas atas  Jumlah outlier  Di antaranya gagal
    Air temperature [K]       293.50      306.30               0                   0
Process temperature [K]       305.35      314.55               0                   0
 Rotational speed [rpm]      1139.50     1895.50               0                   0
         Torque [Nm]          12.80       67.20               0                   0
       Tool wear [min]      -110.50      325.50               0                   0
[grafik tersimpan] 03_boxplot_sesudah.png

Ukuran data setelah preprocessing: (10000, 10)
```

Kondisi data setelah *handling* juga divisualisasikan menggunakan boxplot. Hasil visualisasi ini digunakan untuk melihat perubahan posisi nilai setelah dilakukan pembatasan berdasarkan rentang IQR.



Gambar 3. 3 Boxplot Setelah Handling Outlier

3.3 Transformasi

3.3.1 Penghapusan Kolom

Beberapa kolom dikeluarkan dari dataset karena tidak digunakan sebagai fitur klasifikasi. UDI dan Product ID merupakan atribut identitas, sedangkan Failure Type berpotensi menyebabkan *data leakage* karena berkaitan langsung dengan informasi kegagalan mesin.

```
8   def drop_columns(df: pd.DataFrame) -> pd.DataFrame:
9       subbanner("Drop kolom ID dan kebocoran label")
10      df = df.drop(columns=config.DROP_COLUMNS)
11      print("Kolom dibuang :", config.DROP_COLUMNS)
12      print("Kolom tersisa :", df.columns.tolist())
13      return df
```

```
============================================================
3. TRANSFORMATION
============================================================

--- Drop kolom ID dan kebocoran label ---
Kolom dibuang : ['UDI', 'Product ID', 'Failure Type']
Kolom tersisa : ['Type', 'Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]', 'Target']
```

Setelah proses tersebut dilakukan, dataset hanya mempertahankan atribut yang relevan untuk digunakan sebagai fitur maupun target pada proses klasifikasi.

3.3.2 Encoding Data Kategorikal

Setelah kolom yang tidak diperlukan dikeluarkan, atribut Type masih memiliki nilai dalam bentuk kategori L, M, dan H. Karena proses pembelajaran mesin membutuhkan data dalam bentuk numerik, nilai kategorikal tersebut kemudian diubah menjadi nilai numerik menggunakan metode encoding.

Pada pengerjaan ini digunakan pemetaan L = 0, M = 1, dan H = 2. Proses tersebut dilakukan menggunakan fungsi encode_type() yang telah menggunakan mapping pada config.py. Setelah encoding dilakukan, program juga memeriksa apakah masih terdapat nilai yang tidak berhasil dipetakan.

```python
def encode_type(df: pd.DataFrame) -> pd.DataFrame:
    subbanner(f"Encoding {config.TYPE_COLUMN} {config.TYPE_MAPPING}")
    df = df.copy()
    df[config.TYPE_COLUMN] = df[config.TYPE_COLUMN].map(config.TYPE_MAPPING)
    if df[config.TYPE_COLUMN].isnull().any():
        raise ValueError("Ada nilai Type di luar mapping L/M/H.")
    print(df[config.TYPE_COLUMN].value_counts().sort_index().to_string())
    return df
```

```
--- Encoding Type {'L': 0, 'M': 1, 'H': 2} ---
Type
0    6000
1    2997
2    1003
```

3.3.3 Pemisahan Fitur dan Target

Selanjutnya, dataset dipisahkan menjadi dua bagian, yaitu X sebagai fitur dan y sebagai target. Pemisahan dilakukan agar atribut yang digunakan sebagai dasar prediksi tidak tercampur dengan label yang ingin diprediksi oleh model.

Fungsi separate_xy() digunakan untuk mengambil seluruh kolom selain Target sebagai X, sedangkan kolom Target digunakan sebagai y.

```python
def separate_xy(df: pd.DataFrame):
    subbanner("Penentuan X (fitur) dan y (label)")
    X, y = df.drop(columns=[config.TARGET]), df[config.TARGET]
    print("Fitur X :", X.columns.tolist())
    print("Label y :", config.TARGET)
    print("Bentuk X:", X.shape, "| Bentuk y:", y.shape)
    return X, y
```

```
--- Penentuan X (fitur) dan y (label) ---
Fitur X : ['Type', 'Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']
Label y : Target
Bentuk X: (10000, 6) | Bentuk y: (10000,)
```

Berdasarkan hasil pemisahan tersebut, X berisi fitur kondisi mesin yang akan digunakan untuk melakukan prediksi, sedangkan y berisi label kondisi mesin, yaitu 0 untuk normal dan 1 untuk failure.

3.3.4 Normalisasi MinMaxScaler

Tahap terakhir pada transformasi adalah melakukan normalisasi terhadap fitur menggunakan MinMaxScaler. Normalisasi dilakukan untuk mengubah rentang nilai setiap fitur sehingga berada pada skala yang seragam. Hal ini diperlukan karena setiap fitur memiliki satuan dan rentang nilai yang berbeda, seperti suhu, kecepatan rotasi, torsi, dan waktu penggunaan alat.

Proses normalisasi dilakukan menggunakan fungsi scale_features() dengan MinMaxScaler dari library Scikit-learn. Setelah proses scaling, data hasil transformasi disimpan dalam bentuk DataFrame dengan nama kolom dan indeks yang tetap dipertahankan.

```python
def scale_features(X: pd.DataFrame):
    subbanner("Normalisasi MinMaxScaler")
    scaler = MinMaxScaler()
    X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns, index=X.index)
    print("5 baris pertama setelah scaling:")
    print(X_scaled.head().to_string())
    print("\nMin per kolom:", X_scaled.min().round(2).to_dict())
    print("Max per kolom:", X_scaled.max().round(2).to_dict())
    return X_scaled, scaler
```

```
--- Normalisasi MinMaxScaler ---
5 baris pertama setelah scaling:
   Type  Air temperature [K]  Process temperature [K]  Rotational speed [rpm]  Torque [Nm]  Tool wear [min]
0   0.5             0.304348                 0.358025                0.526460     0.551471         0.000000
1   0.0             0.315217                 0.370370                0.329897     0.615809         0.011858
2   0.0             0.304348                 0.345679                0.453608     0.672794         0.019763
3   0.0             0.315217                 0.358025                0.364261     0.490809         0.027668
4   0.0             0.315217                 0.370370                0.329897     0.500000         0.035573

Min per kolom: {'Type': 0.0, 'Air temperature [K]': 0.0, 'Process temperature [K]': 0.0, 'Rotational speed [rpm]': 0.0, 'Torque [Nm]': 0.0, 'Tool wear [min]': 0.0}
Max per kolom: {'Type': 1.0, 'Air temperature [K]': 1.0, 'Process temperature [K]': 1.0, 'Rotational speed [rpm]': 1.0, 'Torque [Nm]': 1.0, 'Tool wear [min]': 1.0}
```

3.4 Split Data

3.4.1 Pembagian Data Training dan Testing

Pembagian data dilakukan menggunakan fungsi train_test_split() dari Scikit-learn. Pada proses ini digunakan test_size=0.2 untuk menentukan 20% data sebagai data testing, sedangkan 80% sisanya digunakan sebagai data training. Parameter random_state=42 digunakan agar hasil pembagian data tetap konsisten ketika program dijalankan kembali. Selain itu, digunakan stratify=y untuk mempertahankan proporsi kelas target pada data training dan testing.

```python
def split(X: pd.DataFrame, y: pd.Series):
    return train_test_split(X, y, test_size=config.TEST_SIZE, random_state=config.RANDOM_STATE, stratify=y)
```

```
============================================================
4. SPLIT DATA 80:20
============================================================

--- Ukuran data (test_size=0.2, random_state=42, stratify=y) ---
X_train: (8000, 6) (80%)
X_test : (2000, 6) (20%)
y_train: (8000,)
y_test : (2000,)
```

Hasil pembagian menunjukkan bahwa dari keseluruhan 10.000 data, sebanyak 8.000 data digunakan sebagai data training dan 2.000 data digunakan sebagai data testing. Data training selanjutnya menjadi dasar untuk proses resampling, sedangkan data testing disimpan untuk digunakan pada tahap testing dan evaluasi model.

3.4.2 Distribusi Data Training dan Testing

Setelah pembagian dilakukan, distribusi kelas pada data training dan testing diperiksa untuk memastikan bahwa proporsi kelas tetap terjaga. Pemeriksaan dilakukan terhadap y_train dan y_test dengan menampilkan jumlah serta persentase masing-masing kelas.

```python
def report(X_train, X_test, y_train, y_test) -> None:
    subbanner(f"Ukuran data (test_size={config.TEST_SIZE}, "
              f"random_state={config.RANDOM_STATE}, stratify=y)")
    total = len(X_train) + len(X_test)
    print(f"X_train: {X_train.shape} ({len(X_train) / total:.0%})")
    print(f"X_test : {X_test.shape} ({len(X_test) / total:.0%})")
    print(f"y_train: {y_train.shape}")
    print(f"y_test : {y_test.shape}")
    for nama, y in [("y_train", y_train), ("y_test", y_test)]:
        subbanner(f"Distribusi {nama}")
        counts = y.value_counts().sort_index()
        persen = (y.value_counts(normalize=True).sort_index() * 100).round(2)
        print(pd.DataFrame({"jumlah": counts, "persen": persen}).to_string())

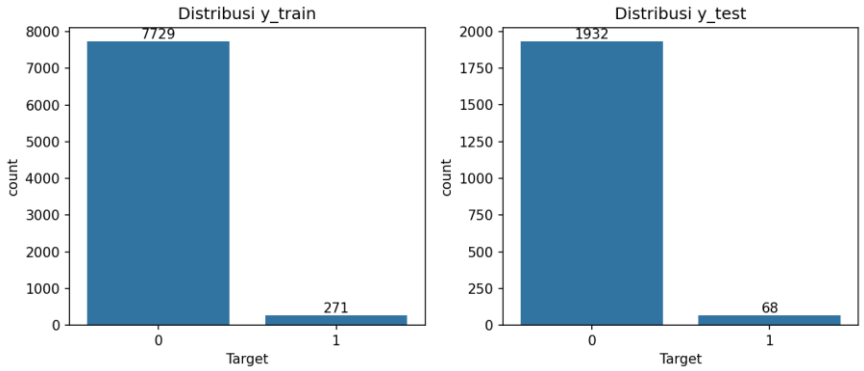

def plot_distribution(y_train, y_test) -> None:
    _, axes = plt.subplots(1, 2, figsize=(9, 4))
    for ax, (judul, data) in zip(axes, [("y_train", y_train), ("y_test", y_test)]):
        sns.countplot(x=data, ax=ax)
        ax.bar_label(ax.containers[0])
        ax.set_title(f"Distribusi {judul}")
    plt.tight_layout()
    save_fig(config.FIGURES_DIR / "04_distribusi_split.png")
```

```
--- Distribusi y_train ---
        jumlah  persen
Target
0         7729   96.61
1          271    3.39

--- Distribusi y_test ---
        jumlah  persen
Target
0         1932    96.6
1           68     3.4
```

Hasil pemeriksaan menunjukkan bahwa kedua bagian data tetap memiliki kelas mayoritas 0 (Normal) dan kelas minoritas 1 (Failure) dengan proporsi yang relatif sama. Hal tersebut sesuai dengan penggunaan stratify=y pada proses pembagian data.

Distribusi tersebut juga divisualisasikan dalam bentuk grafik untuk memperlihatkan perbandingan kelas pada data training dan data testing.



Gambar 3. 4 Grafik Distribusi

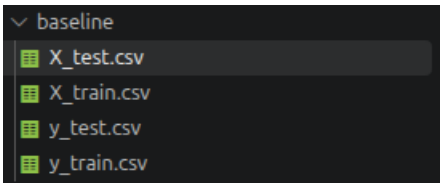
Data testing pada tahap ini tidak mengalami proses resampling. Data tersebut tetap dipertahankan dalam kondisi awal agar nantinya dapat digunakan sebagai data pengujian yang sama untuk eksperimen oversampling maupun undersampling.

3.4.3 Baseline Data

Hasil pembagian data kemudian disimpan sebagai baseline yang akan digunakan bersama dalam eksperimen resampling. Data yang disimpan terdiri dari X_train, X_test, y_train, dan y_test.

Penyimpanan dilakukan menggunakan fungsi save_baseline() sehingga hasil pembagian dapat digunakan kembali tanpa perlu melakukan proses preprocessing dan split ulang. Baseline ini menjadi titik awal yang sama bagi eksperimen oversampling dan undersampling, sehingga kedua pendekatan dapat dibandingkan menggunakan data dasar dan data testing yang sama.

```python
def save_baseline(X_train, X_test, y_train, y_test, directory=config.BASELINE_DIR) -> None:
    directory.mkdir(parents=True, exist_ok=True)
    X_train.to_csv(directory / "X_train.csv", index=False)
    X_test.to_csv(directory / "X_test.csv", index=False)
    y_train.to_csv(directory / "y_train.csv", index=False)
    y_test.to_csv(directory / "y_test.csv", index=False)
    print(f"[baseline tersimpan] {directory}")
```



3.5 Oversampling

3.5.1 Metode Oversampling

Dataset Machine Predictive Maintenance Classification memiliki ketidakseimbangan kelas (*imbalanced data*) yang sangat besar. Pada data training, kelas Failure (minoritas) hanya berjumlah 271 dari total 8000 data atau sekitar 3.39%, sedangkan kelas Normal (mayoritas) mencapai 7729 data. Kondisi ini membuat model cenderung lebih condong memprediksi kelas Normal dan kurang mampu mengenali mesin yang mengalami kegagalan. Untuk mengatasinya, pada eksperimen ini digunakan pendekatan *oversampling*, yaitu menambah jumlah sampel kelas minoritas sehingga distribusi kelas pada data training menjadi lebih seimbang.

Metode *oversampling* yang digunakan adalah Borderline-SMOTE (*Synthetic Minority Over-sampling Technique*) dari library imbalanced-learn. SMOTE bekerja dengan membangkitkan sampel sintetis baru melalui interpolasi antara sebuah sampel kelas minoritas dengan salah satu tetangga terdekatnya yang juga berasal dari kelas minoritas, sehingga sampel baru yang dihasilkan bukan sekadar duplikasi data yang sudah ada. Borderline-SMOTE merupakan varian dari SMOTE yang tidak membangkitkan sampel dari seluruh data minoritas, melainkan hanya dari sampel minoritas yang berada di dekat batas antar kelas (*borderline*), yaitu sampel yang di sekitarnya banyak terdapat data kelas mayoritas. Sampel pada area ini paling sulit diklasifikasikan, sehingga penambahan data sintetis di

sekitarnya membantu model mempelajari batas keputusan antara kelas Normal dan Failure dengan lebih baik.

Parameter yang digunakan pada proses oversampling adalah sebagai berikut:

- sampling_strategy=0.5, yaitu jumlah kelas minoritas dinaikkan hingga mencapai 50% dari jumlah kelas mayoritas (rasio 1:2). Rasio ini dipilih agar jumlah data sintetis tidak terlalu jauh melebihi jumlah data asli kelas minoritas.
- random_state=42, digunakan agar hasil pembangkitan data sintetis konsisten setiap kali kode dijalankan.
- Parameter lainnya menggunakan nilai bawaan library (k_neighbors=5, m_neighbors=10, kind="borderline-1").

Proses *oversampling* hanya diterapkan pada data training. Data testing tidak diubah sama sekali agar evaluasi model tetap dilakukan pada data asli dan tidak terjadi kebocoran data (*data leakage*).

3.5.2 Distribusi Kelas Sebelum Oversampling

Data yang digunakan pada tahap ini adalah data training hasil pemisahan baseline (X_train dan y_train) yang dibaca dari folder output/baseline. Distribusi kelas pada data training sebelum *oversampling* ditunjukkan oleh Tabel 3.2.

```
16    # 1. Load Baseline Data
17    print("Membaca data baseline training dan testing...")
18    X_train = pd.read_csv("output/baseline/X_train.csv")
19    y_train = pd.read_csv("output/baseline/y_train.csv").squeeze()
20    X_test = pd.read_csv("output/baseline/X_test.csv")
21    y_test = pd.read_csv("output/baseline/y_test.csv").squeeze()
22
23    # 2. Cek Distribusi Kelas Sebelum Resampling
24    print("\nDistribusi kelas Target (Sebelum Oversampling):")
25    print(y_train.value_counts())
26
```

Kelas	Jumlah
0 (Normal)	7729
1 (Failure)	271
Total	8000

Tabel 3. 2 Distribusi Kelas Sebelum Oversampling

Berdasarkan tabel di atas, data training sangat didominasi oleh kelas 0 (Normal) sebanyak 7729 sampel atau sekitar 96.61%, sedangkan kelas 1 (Failure) hanya sebanyak 271 sampel atau sekitar 3.39%. Perbandingan kedua kelas adalah sekitar 28.5 : 1, sehingga data training berada dalam kondisi tidak seimbang.

```
Membaca data baseline training dan testing...

Distribusi kelas Target (Sebelum Oversampling):
0    7729
1     271
Name: Target, dtype: int64
```

3.5.3 Implementasi Oversampling

Proses implementasi *oversampling* dilakukan dengan menginisiasi BorderlineSMOTE dari modul imblearn.over_sampling berdasarkan source code berikut:

```
28    # 3. Terapkan Metode Oversampling
29    print("\nMenerapkan BorderlineSMOTE pada data training...")
30    smote = BorderlineSMOTE(sampling_strategy=0.5, random_state=42)
31    X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

Pada kode di atas, objek BorderlineSMOTE dibuat dengan sampling_strategy=0.5 dan random_state=42. Fungsi fit_resample dijalankan hanya pada data training (X_train dan y_train) dan menghasilkan data training baru, yaitu X_train_res dan y_train_res, yang telah ditambah data sintetis kelas minoritas. Data testing (X_test dan y_test) tetap dipertahankan dalam kondisi asli tanpa *resampling* untuk menjaga validitas evaluasi akhir model.

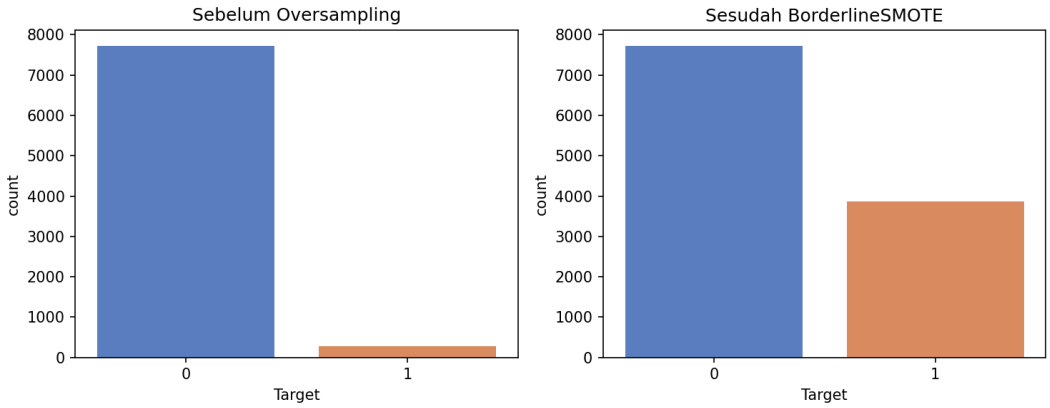
3.5.4 Distribusi Kelas Setelah Oversampling

Setelah BorderlineSMOTE diterapkan, jumlah data kelas minoritas ditambah dengan data sintetis hingga mencapai 50% dari jumlah kelas mayoritas. Distribusi kelas pada data training setelah *oversampling* ditunjukkan oleh Tabel 3.3.

Kelas	Jumlah
0 (Normal)	7729
1 (Failure)	3864
Total	11593

Tabel 3. 3 Distribusi Kelas Setelah Oversampling

Berdasarkan tabel di atas, jumlah data kelas 0 (Normal) tidak berubah yaitu 7729 sampel, sedangkan kelas 1 (Failure) meningkat dari 271 menjadi 3864 sampel. Dengan demikian, terdapat 3593 data sintetis baru yang ditambahkan ke kelas minoritas. Total data training bertambah dari 8000 menjadi 11593 sampel dan proporsi kelas Failure naik dari 3.39% menjadi 33.33% (rasio 2 : 1). Perubahan distribusi ini divisualisasikan pada Gambar 3.5.



Gambar 3. 5 Diagram Perubahan Distribusi

```
Menerapkan BorderlineSMOTE pada data training...

Distribusi kelas Target (Sesudah Oversampling):
0    7729
1    3864
Name: Target, dtype: int64
-> Grafik distribusi disimpan di output/figures/distribusi_oversampling.png
```

3.5.5 Training Decision Tree

Tahap klasifikasi dilakukan menggunakan algoritma Decision Tree yang dilatih pada data training hasil *oversampling* (X_train_res dan y_train_res). Konfigurasi model ditunjukkan pada kode berikut:

```
48  # 5. Training Decision Tree
49  print("\nMelatih model Decision Tree dengan data hasil oversampling...")
50  dt = DecisionTreeClassifier(criterion="entropy", random_state=42)
51  dt.fit(X_train_res, y_train_res)
```

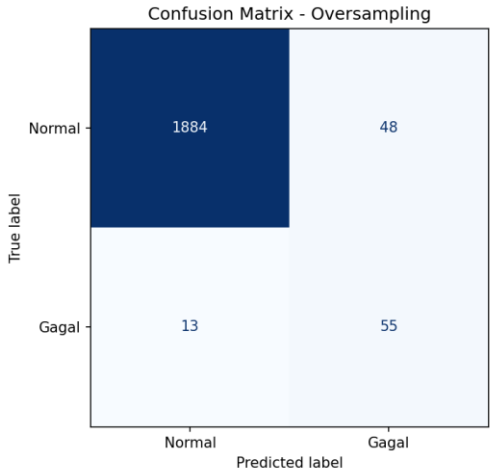
Pada inisialisasi tersebut digunakan parameter criterion="entropy", yaitu pemilihan atribut pemisah pada setiap simpul berdasarkan nilai *information gain* (penurunan entropi). Parameter random_state=42 digunakan agar hasil pelatihan konsisten, sedangkan parameter lainnya menggunakan nilai bawaan sehingga kedalaman pohon tidak dibatasi. Konfigurasi yang sama digunakan pada eksperimen *undersampling* agar perbandingan kedua metode dilakukan pada pengaturan model yang setara. Model kemudian dilatih menggunakan fungsi fit() pada data training yang telah diseimbangkan.

3.5.6 Testing dan Evaluasi

Model Decision Tree diuji menggunakan data testing asli (X_test) yang tidak melalui proses *oversampling*. Data testing berjumlah 2000 observasi yang terdiri dari 1932 data Normal dan 68 data Failure. Proses prediksi dan perhitungan metrik evaluasi dilakukan dengan kode berikut:

```
57  # Menghitung 4 Metrik
58  acc = accuracy_score(y_test, y_pred)
59  prec = precision_score(y_test, y_pred)
60  rec = recall_score(y_test, y_pred)
61  f1 = f1_score(y_test, y_pred)
62
63  print(f"Akurasi  : {acc:.4f}")
64  print(f"Presisi  : {prec:.4f}")
65  print(f"Recall   : {rec:.4f}")
66  print(f"F1-Score : {f1:.4f}")
67
68  print("\nClassification Report:")
69  print(classification_report(y_test, y_pred, target_names=["Normal (0)", "Gagal (1)"]))
70
```

Kinerja model dalam mendeteksi kegagalan mesin direpresentasikan oleh Confusuin matrikx pada Gambar 3.6.



Gambar 3. 6 Confusion Matrix - Oversampling

Berdasarkan matriks tersebut, diperoleh rincian prediksi sebagai berikut;

- True Negative (TN) = 1884
- False Positive (FP) = 48
- False Negative (FN) = 13
- True Positive (TP) = 55

Hasil tersebut kemudian diukur ke dalam metrik evaluasi seperti pada Tabel 3.3.

Metrik	Nilai
Accuracy	96.95%
Precision	53.40%
Recall	80.88%
F1-score	64.33%

Tabel 3. 4 Metrik Evaluasi

```
Melakukan prediksi pada data testing dan evaluasi...
Akurasi  : 0.9695
Presisi  : 0.5340
Recall   : 0.8088
F1-Score : 0.6433
```

Pembahasan Hasil Evaluasi:

Model mencapai Accuracy sebesar 96.95%, yaitu 1939 dari 2000 data testing diprediksi dengan benar. Namun, nilai ini perlu dibaca dengan hati-hati karena data testing didominasi kelas Normal (1932 dari 2000 data atau 96.60%). Model yang selalu menebak Normal pun akan memperoleh accuracy sekitar 96.60%, sehingga accuracy saja tidak cukup menggambarkan kemampuan model dalam mengenali kegagalan mesin dan perlu dilengkapi dengan precision, recall, dan F1-score.

Nilai Precision sebesar 53.40% menunjukkan bahwa dari 103 data yang diprediksi sebagai Failure, sebanyak 55 data benar-benar Failure, sedangkan 48 data lainnya merupakan alarm palsu (False Positive). Nilai Recall sebesar 80.88% menunjukkan bahwa model berhasil mendeteksi 55 dari 68 mesin yang benar-benar mengalami kegagalan, sedangkan 13 mesin lainnya tidak terdeteksi (False Negative). F1-score sebesar 64.33% menggambarkan keseimbangan antara precision dan recall pada pendekatan *oversampling* ini.

Dalam konteks *predictive maintenance*, 13 data False Negative berarti mesin yang sebenarnya akan gagal tidak terdeteksi, sedangkan 48 data False Positive berarti terdapat pemeriksaan mesin yang sebenarnya tidak diperlukan. Perbandingan hasil ini dengan metode *undersampling* dibahas pada subbab 3.7.

3.6 Undersampling

3.6.1 Metode Undersampling

Dataset Machine Predictive Maintenance Classification memiliki masalah ketidakseimbangan kelas (imbalance data) yang signifikan, di mana observasi pada kelas mesin normal (mayoritas) mendominasi secara ekstrem dibandingkan dengan kelas mesin yang mengalami kegagalan (minoritas). Untuk mengatasi ketimpangan

distribusi tersebut, pendekatan Random Undersampling digunakan pada eksperimen ini.

Konsep dari Random Undersampling bekerja dengan cara mengurangi jumlah sampel pada kelas mayoritas secara acak hingga distribusinya seimbang dengan kelas minoritas. Metode ini diterapkan agar model tidak menjadi bias terhadap kelas mayoritas saat fase pelatihan. Parameter sampling_strategy=1.0 digunakan untuk menyeimbangkan kelas menjadi rasio 1:1, dan random_state=42 digunakan untuk menjaga konsistensi hasil pengacakan.

3.6.2 Distribusi Kelas Sebelum Undersampling

Data yang digunakan pada proses ini adalah murni bersumber dari himpunan data latih (training data) hasil pemisahan baseline. Distribusi kelas latih sebelum tahap undersampling ditunjukkan oleh Tabel 3.5.

Kelas	Jumlah
0 (Normal)	7729
1 (Failure)	271
Total	8000

Tabel 3. 5 Distribusi kelas sebelum Undersampling

Berdasarkan tabel di atas, himpunan data latih masih sangat didominasi oleh kelas 0 (Normal) sebanyak 7729 sampel, sedangkan kelas 1 (Failure) hanya sebanyak 271 sampel.

3.6.3 Implementasi Undersampling

Proses implementasi Random Undersampling dilakukan dengan menginisiasi modul dari imbalanced-learn berdasarkan source code berikut:

```python
def run():
    banner("5b. UNDERSAMPLING PADA TRAINING DATA")
    X_train, X_test, y_train, y_test = load_baseline()

    subbanner("Distribusi sebelum undersampling:")
    print(y_train.value_counts())

    # Undersampling process
    rus = RandomUnderSampler(sampling_strategy=1.0, random_state=config.RANDOM_STATE)
    X_train_under, y_train_under = rus.fit_resample(X_train, y_train)

    subbanner("Distribusi sesudah undersampling:")
    print(y_train_under.value_counts())

    # Visualization
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    for ax, (title, data) in zip(axes, [("Sebelum Undersampling", y_train),
                                        ("Sesudah Undersampling", y_train_under)]):
        sns.countplot(x=data, ax=ax)
        ax.bar_label(ax.containers[0])
        ax.set_title(title)
    plt.tight_layout()
    save_fig(config.FIGURES_DIR / "distribusi_undersampling.png")
```

Pada kode di atas, fungsi fit_resample secara ketat hanya diimplementasikan pada data training (X_train dan y_train). Data testing (test set) tetap dipertahankan aslinya (tanpa di-resampling) untuk memastikan validitas evaluasi akhir model.

```
============================================================
5b. UNDERSAMPLING PADA TRAINING DATA
============================================================

--- Distribusi sebelum undersampling: ---
Target
0    7729
1     271
Name: count, dtype: int64

--- Distribusi sesudah undersampling: ---
Target
0    271
1    271
Name: count, dtype: int64
[grafik tersimpan] distribusi_undersampling.png
```

Output program saat memeriksa distribusi kelas pada data training. Sebelum undersampling, kelas 0 (Normal) berjumlah 7.729 data dan kelas 1 (Failure) berjumlah 271 data. Setelah Random Undersampling diterapkan, kedua kelas menjadi seimbang, masing-masing sebanyak 271 data.

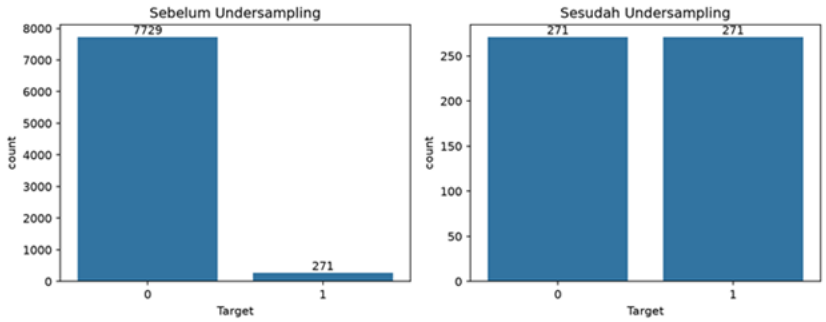
3.6.4 Distribusi Kelas Setelah Undersampling

Setelah metode undersampling diterapkan, kelas 0 (mayoritas) dikurangi secara acak sehingga jumlahnya setara dengan kelas 1. Perbandingan hasil perubahan distribusinya ditunjukkan pada Tabel 3.6.

Kelas	Jumlah
0 (Normal)	271
1 (Failure)	271
Total	542

Tabel 3. 6 Perbandingan hasil perubahan distribusi

Seperti yang terlihat pada tabel, setelah dilakukan reduksi pada kelas mayoritas, jumlah data latih untuk kelas 0 menjadi 271 dan kelas 1 tetap 271 observasi. Secara keseluruhan, total data training menjadi 542 dengan kondisi keseimbangan kelas 1:1, sebagaimana divisualisasikan pada gambar grafik di subbab sebelumnya.



Gambar 3. 7 Grafik Perbandingan

3.6.5 Training Decision Tree

Tahap klasifikasi menggunakan algoritma Decision Tree setelah proses undersampling. Konfigurasi model ditunjukkan pada kode di bawah ini:

```
banner("6. KLASIFIKASI DECISION TREE")
# Konfigurasi disesuaikan dengan eksperimen oversampling anggota kelompok
dt = DecisionTreeClassifier(criterion="entropy", random_state=config.RANDOM_STATE)
print(f"Konfigurasi Model: {dt.get_params()}")
dt.fit(X_train_under, y_train_under)
y_pred = dt.predict(X_test)
```

Pada inisialisasi tersebut, parameter criterion="entropy" digunakan. Konfigurasi ini dibuat konsisten dengan parameter yang diatur pada eksperimen oversampling agar perbandingan akhir klasifikator dapat dilakukan pada kondisi parameter internal yang serupa. Selain itu, parameter random_state=42 turut digunakan. Model lalu dilatih menggunakan fitur latih yang sudah seimbang (X_train_under dan y_train_under).

```
============================================================
6. KLASIFIKASI DECISION TREE
============================================================
h': None, 'max_features': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, '
min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'random_state': 42, 'splitter': 'best'}
```

Gambar 3.10 menampilkan hasil keluaran konfigurasi model Decision Tree yang digunakan pada eksperimen undersampling. Output menunjukkan random_state=42 dan splitter="best", sedangkan parameter lainnya menggunakan nilai bawaan library.

3.6.6 Testing dan Evaluasi

Model Decision Tree dievaluasi menggunakan data testing asli (X_test) yang tidak melalui proses undersampling. Data testing tersebut memuat total 2000 observasi, yang terdiri dari 1932 data Normal dan 68 data Failure. Kinerja kemampuan deteksi model direpresentasikan oleh Confusion Matrix dibawah.

```
banner("7. EVALUASI (PADA DATA TESTING)")
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred):.4f}")
print(f"F1-score  : {f1_score(y_test, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Gambar memperlihatkan source code yang digunakan untuk menghitung Confusion Matrix, Accuracy, Precision, Recall, dan F1-score berdasarkan y_test dan y_pred.

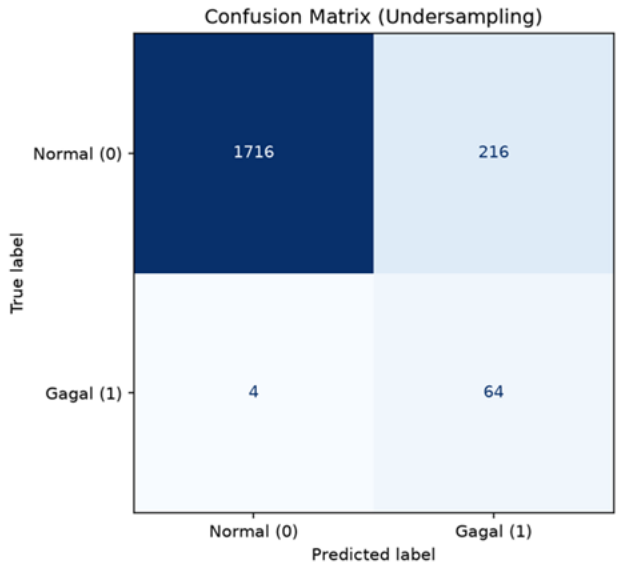
```
============================================================
7. EVALUASI (PADA DATA TESTING)
============================================================
Confusion Matrix:
 [[1716  216]
 [   4   64]]
Accuracy  : 0.8900
Precision : 0.2286
Recall    : 0.9412
F1-score  : 0.3678

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.89      0.94      1932
           1       0.23      0.94      0.37        68

    accuracy                           0.89      2000
   macro avg       0.61      0.91      0.65      2000
weighted avg       0.97      0.89      0.92      2000
```

Gambar menunjukkan hasil evaluasi model pada 2.000 data testing. Confusion Matrix menghasilkan TN=1.716, FP=216, FN=4, dan TP=64, dengan Accuracy 0,8900, Precision 0,2286, Recall 0,9412, dan F1-score 0,3678.



Gambar 3. 8 Confusion Matrix Undersampling

Berdasarkan matriks tersebut, diperoleh rincian prediksi sebagai berikut:

1. True Negative (TN) = 1716
2. False Positive (FP) = 216
3. False Negative (FN) = 4
4. True Positive (TP) = 64

Untuk menilai kualitas klasifikasi secara keseluruhan, hasil matriks diukur ke dalam metrik evaluasi seperti yang tertuang pada Tabel 3.7.

Metrik	Nilai
Accuracy	89.00%
Precision	22.86%
Recall	94.12%
F1-score	36.78%

Tabel 3. 7 Metrik Evaluasi Secara Keseluruhan

Pembahasan Hasil Evaluasi:

Berdasarkan angka aktual hasil eksperimen, model mendeteksi 64 dari total 68 data failure secara tepat pada test set, sedangkan 4 data failure luput (tidak terdeteksi). Hal ini tercermin pada perolehan nilai Recall sebesar 94.12%. Tingginya recall mengindikasikan sensitivitas model yang sangat baik dalam mendeteksi mesin yang gagal.

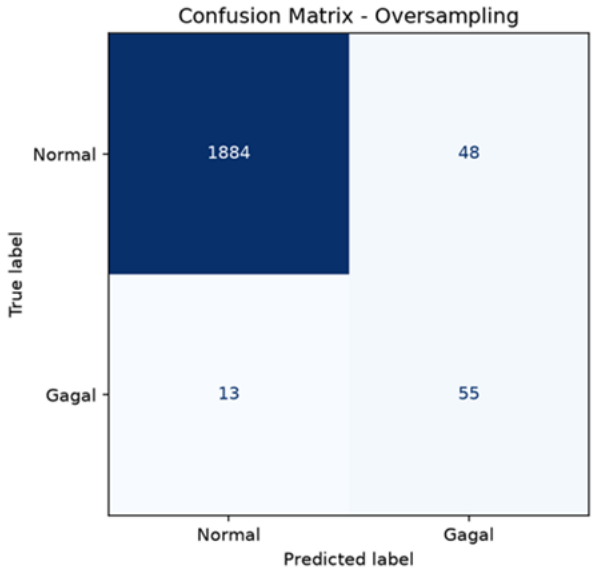
Namun, di sisi lain, Precision model hanya menyentuh angka 22.86%. Nilai precision ini menunjukkan bahwa proporsi kebenaran atas seluruh tebakan failure dari model relatif rendah. Hal ini dipengaruhi secara signifikan oleh adanya 216 alarm palsu (False Positive), di mana data mesin yang aslinya normal secara keliru diprediksi sebagai failure.

Secara menyeluruh, Accuracy model mencapai skor 89.00%, yang berarti proporsi ketepatan prediksi model terhadap dua kelas pada data testing mencapai 89%. Pada akhirnya, F1-score sebesar 36.78% menjadi tolok ukur penggabung yang menggambarkan trade-off antara nilai precision yang rendah dengan recall yang tinggi pada pendekatan undersampling ini.

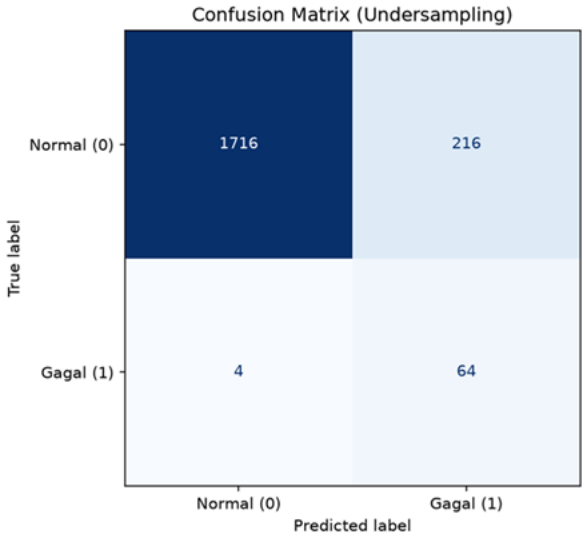
3.7 Perbandingan Hasil

3.7.1 Perbandingan Confusion Matrix

Bagian ini menyajikan perbandingan kinerja model Decision Tree ketika dilatih menggunakan data yang telah melalui proses oversampling (BorderlineSMOTE) dibandingkan dengan undersampling (RandomUnderSampler). Hasil klasifikasi divisualisasikan dalam bentuk Confusion Matrix pada Gambar 3.14 dan Gambar 3.15



Gambar 3. 9 Confusion Matrix Oversampling



Gambar 3. 10 Confusion Matrix Undersampling

Berdasarkan kedua matriks di atas, pola kesalahan dari masing-masing pendekatan dapat diidentifikasi secara kuantitatif. Pada pendekatan Oversampling, model memprediksi 1884 data Normal secara benar (TN) dan 55 data Failure secara benar (TP), dengan jumlah False Positive sebanyak 48 dan False Negative sebanyak 13. Sementara itu, pada pendekatan Undersampling, model memprediksi 1716 data Normal secara benar (TN) dan 64 data Failure secara benar (TP), namun diiringi dengan melonjaknya False Positive menjadi 216 data, sedangkan False Negative menyusut menjadi hanya 4 data.

3.7.2 Perbandingan Accuracy, Precision, Recall, dan F1-Score

Ringkasan komparasi metrik evaluasi dari kedua metode tersebut pada himpunan data uji (test set) dirangkum pada Tabel 3.8

Metode	Accuracy	Precision	Recall	F1-Score
Oversampling	96.95%	53.40%	80.88%	64.33%
Undersampling	89.00%	22.86%	94.12%	36.78%

Tabel 3. 8 Ringkasan Komparasi Metrik Evaluasi

Tabel perbandingan menunjukkan bahwa metode oversampling mencatatkan nilai Accuracy, Precision, dan F1-score yang lebih tinggi secara angka

dibandingkan undersampling. Namun, di sisi lain, metode undersampling memperoleh nilai Recall yang lebih tinggi, yang menandakan bahwa tingkat kepekaan model dalam mendeteksi observasi failure meningkat secara drastis saat menggunakan metode tersebut.

3.7.3 Pembahasan Hasil

1. Accuracy

   Hasil evaluasi menunjukkan bahwa pendekatan oversampling mencapai tingkat Accuracy (96.95%) yang lebih besar dibandingkan undersampling (89.00%). Hal ini berkaitan erat dengan kemampuan model oversampling yang lebih stabil dalam memprediksi data kelas mayoritas tanpa banyak memicu kesalahan identifikasi.

2. Precision

   Nilai Precision pada eksperimen oversampling tercatat sebesar 53.40%, yang jauh lebih tinggi dibandingkan hasil undersampling yang hanya 22.86%. Tingginya angka precision pada model oversampling menandakan bahwa rasio tebakan benar terhadap seluruh tebakan 'gagal' pada model tersebut jauh lebih meyakinkan dibandingkan model undersampling yang kerap membangkitkan banyak peringatan gagal keliru (alarm palsu).

3. Recall

   Berbeda dengan precision, metrik Recall justru dimenangkan oleh model dengan undersampling (94.12%), berbanding 80.88% pada model oversampling. Ini membuktikan bahwa reduksi pada kelas mayoritas melalui undersampling membuat klasifikator sangat berfokus dalam mengenali kelas failure (minoritas), sehingga angka deteksinya sangat kuat.

4. F1-score

   Metrik F1-score pada model oversampling mencapai 64.33%, menunjukkan titik ekuilibrium (keseimbangan) yang lebih moderat antara Recall dan Precision. Sementara pada undersampling, nilai F1-score hanya bertengger di

36.78%, dipicu oleh ketimpangan jarak antara Recall yang sangat tinggi dan Precision yang sangat rendah.

5. Confusion Matrix

Perbedaan nilai metrik di atas sangat tergambar jelas pada pola pembagian di dalam Confusion Matrix. Model undersampling berani menebak gagal secara masif (menekan False Negative hingga sisa 4 data), namun konsekuensinya adalah lonjakan False Positive hingga 216 data, yang merugikan presisi. Sebaliknya, model oversampling lebih berhati-hati dalam menembak kelas failure (menekan False Positive menjadi 48 data saja), namun konsekuensi yang harus dibayar adalah 13 mesin berstatus kegagalan tidak berhasil dikenali (False Negative). Dalam konteks sistem peringatan kerusakan (predictive maintenance), kedua pola kesalahan klasifikasi ini menyajikan bentuk toleransi ketahanan mesin yang berlainan berdasarkan prioritas sistem di dunia nyata.

BAB IV

KESIMPULAN

Berdasarkan serangkaian tahapan yang telah dilakukan pada dataset *Machine Predictive Maintenance Classification*, proses pengolahan data awal telah berhasil mempersiapkan 10.000 observasi dengan baik untuk tahap pemodelan. Proses pembersihan data membuktikan tidak adanya *missing value* maupun data duplikat, sementara penanganan *outlier* dieksekusi menggunakan metode *clipping* berdasar batas *Interquartile Range* (IQR) sehingga tidak ada data yang terbuang. Selanjutnya, atribut yang berpotensi menyebabkan kebocoran data (*data leakage*) seperti *Failure Type*, beserta kolom identitas, telah dihapus. Fitur yang tersisa kemudian ditransformasikan melalui *encoding* untuk data kategorikal dan normalisasi *MinMaxScaler*, sebelum akhirnya dibagi menjadi himpunan *training* dan *testing* dengan rasio 80:20 secara proporsional (*stratified*).

Dalam mengatasi tantangan ketidakseimbangan kelas (*imbalanced data*), eksperimen ini membandingkan kinerja algoritma Decision Tree yang dilatih menggunakan dua pendekatan *resampling* yang berbeda, yakni *oversampling* dan *undersampling*. Pada pendekatan *oversampling* yang menggunakan metode Borderline-SMOTE, model menunjukkan kinerja evaluasi yang jauh lebih seimbang dan stabil. Model ini sanggup mencapai tingkat akurasi sebesar 96,95% dan F1-Score 64,33%. Keunggulan utama dari model *oversampling* ini terletak pada presisinya (53,40%) yang jauh lebih baik karena berhasil menekan jumlah alarm palsu (*False Positive*) menjadi hanya 48 observasi.

Di sisi lain, pendekatan *undersampling* yang menggunakan *RandomUnderSampler* menghasilkan klasifikator dengan tingkat sensitivitas yang sangat tajam dalam mendeteksi ancaman kegagalan mesin. Hal ini tercermin dari perolehan nilai *Recall* yang sangat superior di angka 94,12%, di mana model tersebut hanya melewatkan 4 data kegagalan mesin (*False Negative*) dari total keseluruhan pengujian. Meskipun deteksinya sangat kuat, keunggulan ini memicu penurunan drastis pada metrik Presisi (22,86%) dan Akurasi (89,00%) akibat tingginya tebakan gagal yang keliru, mencapai 216 peringatan palsu.